Analysis and Design of Algorithms

CS3230

Recitation

Week 9

Prof Lee Wee Sun

Dynamic Programming

Hallmarks of dynamic programming

- Optimal substructure
- Overlapping subproblems

- Optimal substructure
  - Optimal solution contains optimal solution of subproblems
- Think of the optimal solution
  - Can it be decomposed into optimal subproblems that can be combined together to form the optimal solution?
  - Write the recurrence: optimal solution in terms of optimal subproblems

- Check for overlapping subproblems
  - Are some subproblems solved over and over again?
  - How many subproblems are there?
- If there are enough overlapping subproblems, consider a memoization or dynamic programming solution.

- In memoization, compute solution **recursively** only for **the first time** and **store** it, **otherwise look up solution**.
- For bottom-up dynamic programming, write an iterative program, taking care that a **subproblem** has been **solved before** it is **used**.

Runtime

Each subproblem is solved once, so runtime can be bounded by
O(no. subproblems * work done in the subproblem).

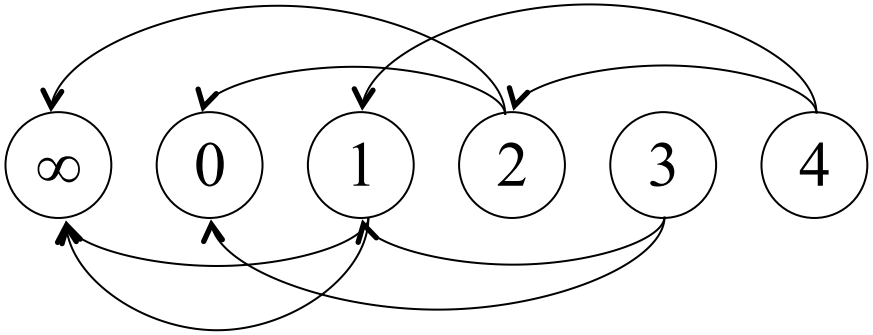
Subproblem Graph

A subproblem graph is often useful for visualizing a dynamic programming algorithm

- It is essentially a recursion tree where all nodes for the same subproblem are collapsed together.

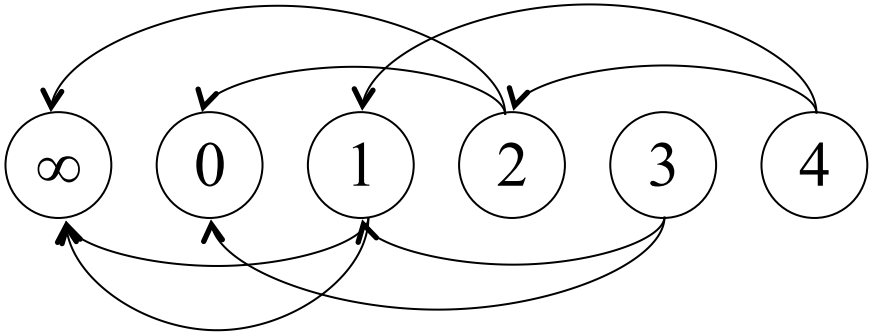
Consider the recurrence for the coin changing problem

$$M(i) = \begin{cases} 1 + \min_{1 \le j \le k} \{M(i - d_j)\} & \\ 0 & i = 0 \\ \infty & i < 0 \end{cases}$$



Example: $n = 4$, denominations 2, 3

- Directed edge from subproblem x to y if x requires solution for y
- Bottom up dynamic programming does reverse topological sort of graph before solving subproblems so that subproblems that are required are already solved
- Top down memoization method does a depth first search of the graph



Each subproblem solved at most once, so runtime is number of nodes times running time in the subproblem.

Running time in subproblem usually proportional to number of edges.

Running time usually linear in number of vertices and edges.

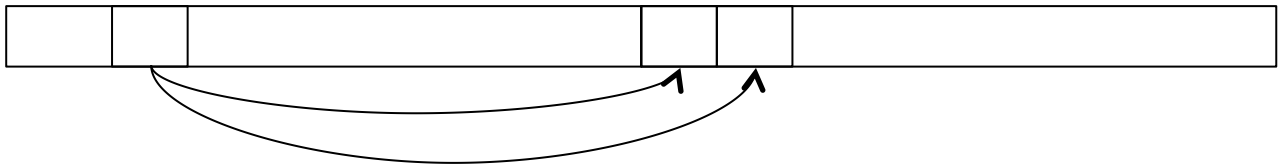
Midterm Q2

- Find max subseq sum in $A[1..n]$ using divide and conquer

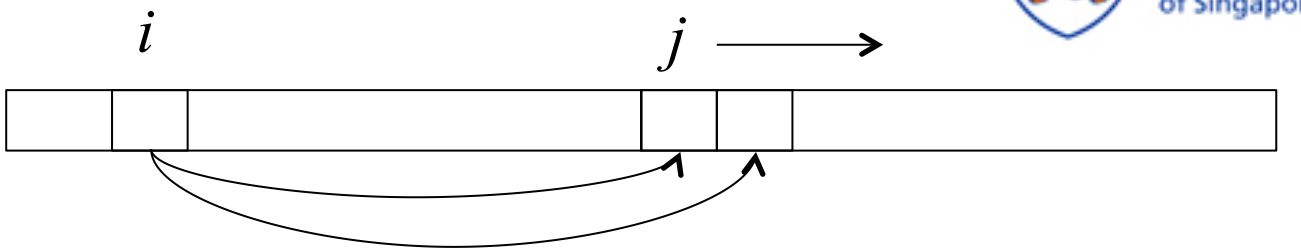
$$\max_{i,j} \sum_{k=i}^{j} A[k]$$

- Problem solving strategy: Try simplest solution first
  - Help to understand problem
  - Get initial, maybe slow, but correct solution!
  - Can iteratively improve: small improvements require less creativity

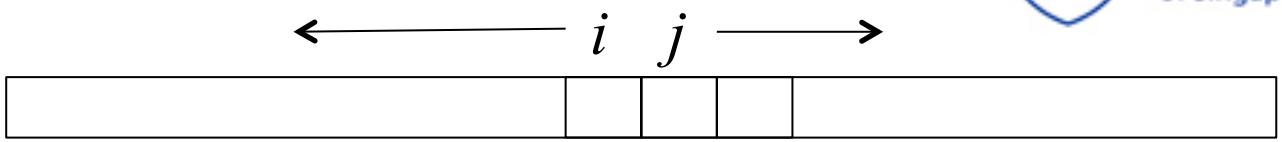
- Main idea: divide into left subproblems and right subproblems.
  - Get max subseq sum on left subarray and right subarray
  - But also need to consider subarrays that start on left and end on right



- Initial simple method: Find all pairs with i from left subarray and j from right subarray and sum.
  - Two **for** loops for i and j: $\Theta(n^2)$
  - Sum from i to j for each pair: $\Theta(n)$
  - Total: $\Theta(n^3)$ for combine step
- Recurrence: $T(n) = 2T(n/2)+\Theta(n^3) = \Theta(n^3)$ by case 3 of Master Theorem



- Improved method: Fix i from left subarray and iteratively compute sum for j from *mid* to end of array.
  - Two **for** loops for i and j: $\Theta(n^2)$
  - Iteratively add sum to j for each pair: $\Theta(1)$ for each j. Total: $\Theta(n^2)$ for combine step
- Recurrence: $T(n) = 2T(n/2)+\Theta(n^2) = \Theta(n^2)$ by case 3 of Master Theorem



- Further improvement: Compute sum for both $A[i..mid]$ and $A[mid+1..j]$ from the *mid* to left and right and sum up $A[i..mid]+A[mid+1..j]$.
  - Compute running sums and largest running sum from *mid*: $\Theta(n)$
- Recurrence: $T(n) = 2T(n/2)+\Theta(n) = \Theta(n \log n)$ by case 2 of Master Theorem

Linear time dynamic programming solution:

- Optimal substructure
  - Let $M(j)$ be the max subseq sum for subseq that ends at i:

$$M(j) = \max_i \sum_{i=1}^{j} A[k]$$

  - Then

$$M(j) = \begin{cases} A[j] & \text{if } M(j-1) \text{ is negative} \\ M(j-1) + A[j] & \text{otherwise} \end{cases}$$

  - If longer than single element, must be extended (towards $i = 1$) using max subseq sum that ends at i-1, otherwise can cut and paste better solution.

$$M(j) = \begin{cases} A[j] & \text{if } M(j-1) \text{ is negative} \\ M(j-1) + A[j] & \text{otherwise} \end{cases}$$

- Use array $M[1..n]$ to store $M(j)$.
- Linear time to compute by iterating from 1 to n.
- Go through array again to find max over j of $M(j)$ in linear time.

Common DP Decompositions

If we only take small steps, may miss solutions that are quite different, e.g. the DP rather than divide and conquer solution for max subseq sum.

- Learn many (different) techniques, divide & conquer, DP, greedy, etc., then can see if any of them applies to the problem
- For DP, can learn many decompositions, then can see if any of them applies to the problem.

Common decompositions you have seen:

- Formulate as opt solution up to position (or step, or value, etc.) i, decomposed in terms of opt soln up to position i-1 or some $k < i$.
  - *Examples*: max subseq sum, coin changing, Bellman-Ford shortest path algorithm (from CS2010, CS2020)
- Formulate as opt soln up to a pair (i,j), decomposed in terms of optimal soln of $(i-k,j)$, $(i, j-k)$, for some value k.
  - *Example*: Longest common subsequence

- Formulate as opt solution using elements in $\{1,.., k\}$, decomposed in terms of opt soln using elements in $\{1,.., k\text{-}1\}$.
  - *Examples*: knapsack, Floyd-Warshall all-pair shortest path (sometimes covered in CS2010, CS2020)
- Formulate as opt solution for an interval, decomposed in terms of optimal solution for sub-intervals.
  - *Example*: Matrix chain multiplication (optional material)

Admin

- Short survey to get feedback (Survey on sidebar on Coursemology): 100 EXP. By this Sunday.
- Reminder: You may discuss, but write up your homeworks yourself. If you cannot write up yourself, you do not understand. If you have difficulties, see teaching staff for consultation:
  - Tues 12noon-1pm (AS6 5-04 Lee),
  - Tues 2-3pm (DR1 Oh),
  - Wed 2-3pm (DR1 Hou),
  - Fri 3-4pm (DR1 Li),
  - Fri 6-7pm (COM1 B103 Purwanto)

MCQ

Question 1

[Longest increasing subsequence] Given an array A of N distinct integers. We want to find the length of the longest increasing subsequence of A.

For example: $A = \{10, 30, 50, 20, 60, 40, 15\}$. Some increasing subsequence of A are: $\{10, 20\}$; $\{10, 30, 50, 60\}$; $\{20, 40\}$; $\{10, 20, 40\}$;... The longest increasing subsequence for the above example has length 4.

Let $LIS(k)$ be the length of the longest increasing subsequence of A in $A[1..k]$ subject to $A[k]$ being the last number of the sequence.

For the above example: $LIS(4) = 2$ because the longest subsequence that ends at $A[4] = 20$ is $\{10, 20\}$.

Which is the correct recursive formula to compute $LIS(k)$, (ignoring boundary conditions)?

- $LIS(k) = \begin{cases} 1 + LIS(k-1), & \text{if } k > 1 \text{ and } A[k] > A[k-1] \\ 1, & \textit{otherwise} \end{cases}$
- $LIS(k) = 1 + \max\{LIS(u) : u = 1, \ldots, k-1 \text{ and } A[u] < A[k]\}$
- None of these formulas is correct.

Answer: B

LIS(k) = 1+ max{LIS(u): u = 1, .. , k-1 and $A[u] < A[k]$} (assume max of empty set is 0)

To see this, an optimal solution that ends at k can be decomposed into the $A[k]$ and earlier subsequence. The earlier subsequence must be optimal, otherwise by cut and paste, we get contradiction. There are k-1 possible endings for the earlier subsequence and we take the max of them.

Question 2

[Rod Cutting Problem] Given a rod of length n inches and a table of prices p_i for $i = 1, 2, ..., n$ inches, determine the maximum revenue r_n obtained by cutting up the rod of length n inches and selling the pieces.

Which of the following gives the correct recurrence for solving the Rod Cutting Problem?

- $r_n = p_i + r_{n-i}$
- $r_n = r_i + r_{n-i}$
- $r_n = \max_{1 \le i \le n}(p_i + r_{n-i})$

Answer: C

$r_n = \max_{1 \le i \le n} (p_i + r_{n-i})$

To see this, note that the optimal solution can be decomposed into the first piece of cost p_i and the remainder which is further cut. The remainder must be cut optimally, otherwise by cut and paste we can get a better solution overall. There are n possible positions to cut the first piece, so we take the max of the positions.

Question 3

Problem: We have an m-by-n grid with some vertices blocked. Starting from the top left corner, suppose we can only move Down or Right. How many possible ways to reach the bottom right corner of the grid? Let $N(i,j)$ be the number of ways to reach the point (i,j), where $(0,0)$ is the top-left corner. What is the recursive formula for $N(i,j$?

- $N(i,j) = \begin{cases} 0, & \text{if } (i,j) \text{ is blocked} \\ \max(N(i-1,j), N(i,j-1)), & \text{if } (i,j) \text{ is unblocked} \end{cases}$

- $N(i,j) = \begin{cases} 0, & \text{if } (i,j) \text{ is blocked} \\ N(i-1,j) + N(i,j-1), & \text{if } (i,j) \text{ is unblocked} \end{cases}$

- $N(i,j) = \begin{cases} 0, & \text{if } (i,j) \text{ is blocked} \\ N(i-1,j-1) + 1, & \text{if } (i,j) \text{ is unblocked} \end{cases}$

- $N(i,j) = \begin{cases} 0, & \text{if } (i,j) \text{ is blocked} \\ N(i-1,j) + N(i,j-1) + 1, & \text{if } (i,j) \text{ is unblocked} \end{cases}$

Solution

$$N(i,j) = \begin{cases} 0, & \text{if } (i,j) \text{ is blocked} \\ N(i-1,j) + N(i,j-1), & \text{if } (i,j) \text{ is unblocked} \end{cases}$$

Answer: B

The only ways to get to (i,j) is through $(i\text{-}1,j)$ or $(i,j\text{-}1)$. Summing the total number of ways through these two vertices gives the total number of ways to get to (i,j).

Note that we often take max/min when we want the maximize/minimize but the same idea works when we want to sum.

Question 4

Consider using merge sort to sort an array of n distinct elements. Will using memoization give a substantial speedup for merge sort?

- Yes, we do not need to recompute any subproblem that has been solved before.
- No, computing all possible subproblems will take roughly n! time.
- No, there is no overlapping subproblem.

Solution

Answer: C

There are no overlapping subproblems (same contagious subsequence that needs to be sorted) as all the elements are distinct.

Note that we don't actually need to solve $n!$ permutations of the array, only the $O(n)$ that appear as part of the recursion tree.

Question 5

Who is the Master of Algorithms pictured below?

- Don Knuth
- Richard Bellman
- Lester Ford Jr.
- D.R. Fulkerson

Question 5

Richard Bellman

Often credited for dynamic programming.
Known for Bellman-Ford algorithm for shortest path, Bellman equation (in optimal control), curse of dimensionality, ...

Structured Questions

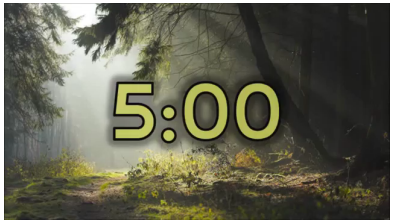
Question 1

You are going on a trip and there are n hotels at increasing distances $d_1, \ldots, d_n$ from start costing $c_1, \ldots, c_n$ along the way. You can travel at most 200km a day and then need to stay overnight at a hotel. Assume that you end at the hotel at d_n without needing to stay at that hotel. What is the cheapest cost for the trip?

Question 1(a)

You are going on a trip and there are n hotels at increasing distances $d_1, \ldots, d_n$ from start costing $c_1, \ldots, c_n$ along the way. You can travel at most 200km a day and then need to stay overnight at a hotel. Assume that you end at the hotel at d_n without needing to stay at that hotel. What is the cheapest cost for the trip?

Write the recurrence for the problem. For simplicity, assume consecutive hotels within 200km of each other.



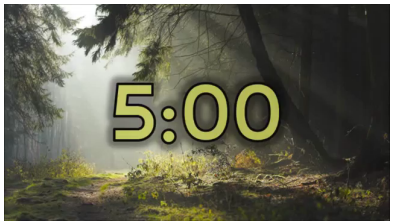
You are going on a trip and there are n hotels at increasing distances $d_1, \ldots, d_n$ from start costing $c_1, \ldots, c_n$ along the way. You can travel at most 200km a day and then need to stay overnight at a hotel. Assume that you end at the hotel at d_n without needing to stay at that hotel. What is the cheapest cost for the trip?

Solution: Consider an optimal solution. If we remove the destination hotel, we still have a set of hotels with the last hotel at d_i. This set must form the optimal way to get to d_i, otherwise we can get a better solution to the original problem by replacing it with the optimal solution. Let $C(j)$ be the optimal cost to get to d_j. Then

$$C(j) = \begin{cases} 0 & \text{if } d_j \leq 200 \\ \min\{c_i + C(i) : i < j, d_j - d_i \leq 200\} & \text{otherwise} \end{cases}$$

Question 1(b)

Write a memoized solution for the problem. What is its runtime?



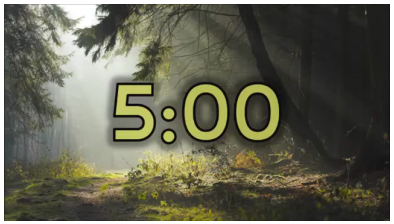
Solution: The minimum costs to reach the hotels are stored in $C[1\,..\,n]$ and initialized to -1. Assume the distances are in $d[1..n]$, and the hotel costs are in $c[1..n]$.

```
MinCost(j)
if d[j] ≤ 200
        C[j] = 0
elseif C[j] == −1
        C[j] = min{c[i] + MinCost(i) : i < j, d[j] − d[i] ≤ 200}
return C[j]
```

There are $O(n)$ subproblems. In each subproblem do $O(n)$ amount of work in finding the minimum number of hotels within 200 may be $O(n)$. Hence runtime is $O(n^2)$.

Question 1(c)

Write a bottom-up dynamic programming pseudo-code for the problem and give the runtime.



Write a bottom-up dynamic programming pseudo-code for the problem and give the runtime.

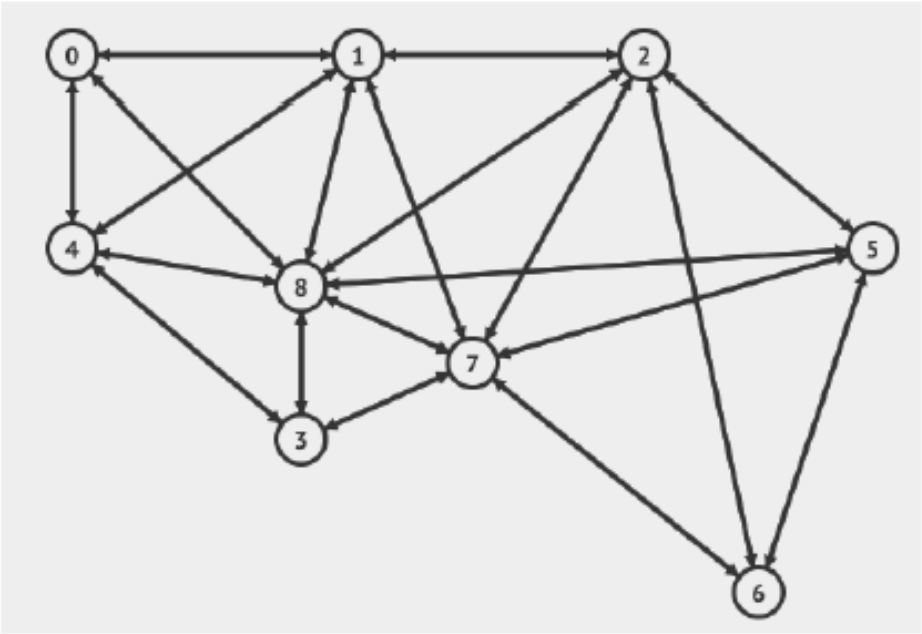
Solution:

```
for j = 1 to n
    if d[j] ≤ 200
        C[j] = 0
    else
        C[j] = min{c[i] + C[i] : i < j, d[j] − d[i] ≤ 200}
```

Runtime: loop n times, each time $O(n)$, so $O(n^2)$

The Travelling Salesman Problem is NP-Hard and hence we should not be looking for a polynomial-time exact solution to solve the problem. Still, this has not discouraged our amazing salesman, Mr Mai Buchu, from trying to find the shortest tour in his country. He would like to visit every city and sell his latest product that will definitely go viral!

More formally, we are given a directed weighted graph $G = (V, E)$ with n vertices, in which each vertex represents a city in the country. An edge (u, v) is in G if it is possible to travel from city u to city v. Given a starting city s, we are required to compute the lowest cost of the tour that starts from s, goes through all other $n - 1$ cities exactly once, and returns to s at the end of the tour. An example of the graph is shown below (without the weights).



Question 2

Assume that the number of cities in the country is small (less than 20 cities). Can you help Mr Mai to come up with the best tour in the country?

Question 2(a)

Suggest a complete search algorithm (that does not use dynamic programming) and the runtime for that algorithm.

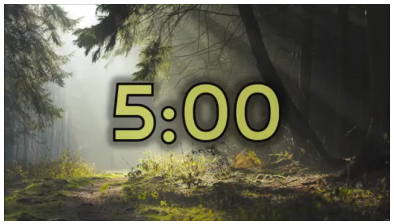
Solution: We can check the cost for all permutations of the tour, and then select the tour with the minimum cost. We can use depth first search/recursion to iterate all permutations. Since the first node is fixed, there are $(n-1)!$ permutations that we have to consider. Computing the cost of a permutation takes $O(n)$ time. The total time take is $O(n \times (n-1)!) = O(n!)$.

Notes: Take note that $O(n!) \neq O((n-1)!)$. Depending on the implementation of depth first search/recursion, the runtime might be different, but for the purpose of this question we will overlook the differences in implementation and accept a factorial time solution.

Question 2(b)

Notice that in the complete search algorithm, the computation of subtours is overlapping (e.g. suppose the first four cities in two different tours are $A - B - C - D$ and $A - C - B - D$, then the computation of the remainder of the tour is an overlapping subproblem). Use this observation and state a recurrence relation that can be used to obtain a dynamic programming or memoization algorithm for this problem.

(Hint: One possible recurrence can be formed by defining $TSP(pos, remaining)$ to compute the shortest distance to visit the remaining cities starting from current city pos, and then return to the starting city s.)



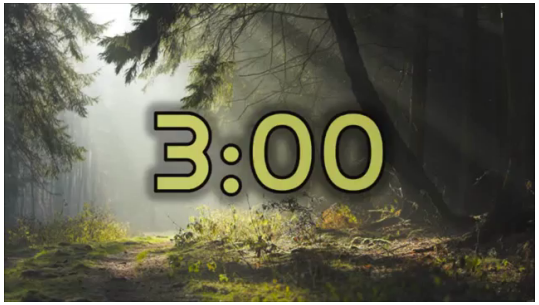
Solution: Our DP state will contain 2 parameters: set of remaining cities to visit and current position. $TSP(pos, remaining)$ will compute the shortest distance to visit the remaining cities starting from pos, and then return to the starting city s. To go from pos to s, we need to move to a next city v, then go through the remaining cities from v before returning to s. Going through the remaining cities from v before returning to s must be done optimally, otherwise we can replace the solution with the optimal one and obtain a better solution, contradicting the fact that $TSP(pos, remaining)$ is optimal. This gives:

$$TSP(pos, remaining) = \begin{cases} dist(pos, s) \text{ if } remaining \text{ contains no city} \\ min(dist(pos, v) + TSP(v, remaining - \{v\})) \\ \qquad \text{for all } v \text{ in } remaining \end{cases}$$

$TSP(s, V - s)$ will return the optimal cost of the tour.

Question 2(c)

What is the asymptotic runtime of the dynamic programming or memoized solution to TSP?



Solution: There are $O(n \times 2^n)$ distinct states since there are n vertices and there are 2^{n-1} distinct subsets in a set of $n-1$ elements (not including the start vertex). The computation in each state can be done in $O(n)$ time. Hence, a dynamic programming or memoized solution takes $O(n^2 2^n)$ time.